5G-SMART DIABETES TOWARD PERSONALIZED DIABETES DIAGNOSIS WITH HEALTHCARE BIG DATA CLOUDS

A "Project Stage II" Report submitted to
JNTU Hyderabad in partial fulfilment
of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

LALIT SIRVI	**22S11A6625**
KILLARI NARENDRA	**22S11A6630**
NUKALA SHODITHA	**22S11A6646**
BATHULA SNIGDHA	**22S11A6652**

Under the Guidance of

E. MOTHILAL
B. Tech, M. Tech
Assistant Professor Of CSE (AI&ML)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
MALLA REDDY INSTITUTE OF TECHNOLOGY & SCIENCE
An UGC Autonomous Institution
(Approved by AICTE New Delhi and Affiliated to JNTUH)
(Accredited by NBA & NAAC with "A" Grade)
An ISO 9001: 2015 Certified Institution
Maisammaguda, Medchal (M), Hyderabad-500100, T. S.

MARCH 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)
MALLA REDDY INSTITUTE OF TECHNOLOGY & SCIENCE
An UGC Autonomous Institution
(Approved by AICTE New Delhi and Affiliated to JNTUH)
(Accredited by NBA & NAAC with "A" Grade)
An ISO 9001: 2015 Certified Institution
Maisammaguda, Medchal (M), Hyderabad-500100, T. S.
MARCH 2026

CERTIFICATE

This is to certify that the Project Stage II entitled **"5G-SMART DIABETES TOWARD PERSONALIZED DIABETES DIAGNOSIS WITH HEALTHCARE BIG DATA CLOUDS"** has been submitted by **LALIT SIRVI (22S11A6625), KILLARI NARENDRA (22S11A6630), NUKALA SHODITHA (22S11A6646) and BATHULA SNIGDHA (22S11A6652)** in partial fulfillment of the requirements for the award of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE & ENGINEERING (AI&ML)**. This record of bonafide work carried out by them under my guidance and supervision. **The result embodied in this Project Stage II report has not been submitted to any other University or Institute for the award of any degree.**

Mr. E. MOTHILAL
Assistant Professor
CSE (AI&ML)
Project Guide

Dr. MAITHILI DEVIREDDY
Head of the Department
CSE (AI&ML)

External Examiner

ACKNOWLEDGEMENT

The Project Stage II work carried out by our team in the Department of CSE (Artificial Intelligence and Machine Learning), Malla Reddy Institute of Technology and Science, Hyderabad. ***This work is original and has not been submitted in part or full for any degree or diploma of any other university.***

We wish to acknowledge our sincere thanks to our project guide **Mr. E. Mothilal** Assistant Professor of CSE (Artificial Intelligence and Machine Learning) for formulation of the problem, analysis, guidance and his continuous supervision during the course of work.

We acknowledge our sincere thanks to **Dr. Vaka Murali Mohan,** Professor & Principal, Department of Computer Science and Engineering and **Dr. Maithili Devireddy**, Head of the Department and Coordinator, Department of Computer Science and Engineering (AI&ML), faculty members of CSE (Artificial Intelligence and Machine Learning) Department for their kind cooperation in making this Project work a success.

We extend our gratitude to **Sri. Ch. Malla Reddy**, Founder Chairman MRGI and **Sri. Ch. Mahender Reddy**, Secretary MRGI, **Dr. Ch. Bhadra Reddy**, President MRGI, **Sri. Ch. Shalini Reddy**, Director MRGI, **Sri. P. Praveen Reddy**, Director MRGI, for their kind cooperation in providing the infrastructure for completion of our Project Stage II.

We acknowledge our special thanks to the entire teaching faculty and non-teaching staff members of the Computer Science & Engineering (Artificial Intelligence and Machine Learning) Department for their support in making this project work a success.

LALIT SIRVI	**22S11A6625**	_____
KILLARI NARENDRA	**22S11A6630**	_____
NUKALA SHODITHA	**22S11A6646**	_____
BATHULA SNIGDHA	**22S11A6652**	_____

INDEX

LIST OF FIGURES

ABSTRACT

Data science methods have the potential to benefit other scientific fields by shedding new light on common questions. One such task is help to make predictions on medical data. Diabetes mellitus or simply diabetes is a disease caused due to the increase level of blood glucose. Various traditional methods, based on physical and chemical tests, are available for diagnosing diabetes. The methods strongly based on the data mining techniques can be effectively applied for high blood pressure risk prediction. In this project, we explore the early prediction of diabetes via five different data mining methods including: GMM, SVM, Logistic regression, ELM, ANN. The experiment result proves that ANN (Artificial Neural Network) provides the highest accuracy than other techniques.

The study focuses on collecting and preprocessing patient data, including parameters such as age, BMI, glucose level, blood pressure, insulin, and family history. These attributes are crucial for building a reliable prediction model. Data preprocessing techniques such as normalization, missing value treatment, and feature selection are performed to ensure data quality and consistency. The cleaned dataset is then used to train and test multiple machine learning algorithms to identify the most efficient model for diabetes prediction.

The performance of each algorithm is evaluated using key metrics such as accuracy, precision, recall, and F1-score. Comparative analysis of the results reveals that while all algorithms demonstrate satisfactory prediction ability, the Artificial Neural Network outperforms others due to its capability to capture complex, nonlinear relationships among variables. The proposed approach demonstrates the usefulness of machine learning in the healthcare domain, providing a foundation for developing automated, accurate, and efficient diabetes prediction systems that can assist doctors in early diagnosis and treatment planning.

CHAPTER- 1. INTRODUCTION

Human body needs energy for activation. The carbohydrates are broken down to glucose, which is the important energy source for human body cells. Insulin is needed to transport the glucose into body cells. The blood glucose is supplied with insulin and glucagon hormones produced by pancreas. Insulin hormones produced by the beta cells of the islets of Langerhans and glucagon hormones are produced by the alpha cells of the islets of Langerhans in the pancreas. When the blood glucose increases, beta cells are stimulated and insulin is given to the blood.

So blood glucose is kept in a narrow range. Diabetes is a chronic disease with the potential to cause a worldwide health care crisis. According to International Diabetes Federation 382 million people are living with diabetes across the whole world. By 2035, this will be doubled as 592 million. However, early prediction of diabetes is quite challenging task for medical practitioners due to complex interdependence on various factors. Data mining is a process to extract useful information from large database, as there are very large and enormous data available in hospitals and medical related diabetes. It is a multidisciplinary field of computer science which involves computational process, machine learning, statistical techniques, classification, clustering and discovering patterns.

Recently, Data mining techniques have been widely used in predicting the data like time-series. A number of data mining algorithms have been proposed for early prediction of disease with higher accuracy in order to save human life and reduce the treatment cost . Thus, applying these algorithms to predict diabetes should be done. In our work, we used five different supervised learning methods to conduct our experiment.

CHAPTER -2. LITERATURE SURVEY

K. Vijiya Kumar et al. proposed random Forest algorithm for the Prediction of diabetes develop a system which can perform early prediction of diabetes for a patient with a higher accuracy by using Random Forest algorithm in ma- chine learning technique. The proposed model gives the best results for diabetic prediction and the result showed that the prediction system is capable of predicting the diabetes disease effectively, efficiently and most importantly, instantly.

Muhammad Azeem Sarwar et al. proposed study on prediction of diabetes using machine learning algorithms in healthcare they applied six different machine learning algorithms Performance and accuracy of the applied algorithms is discussed and compared. Comparison of the different machine learning techniques used in this study reveals which algorithm is best suited for prediction of diabetes. Diabetes Prediction is becoming the area of interest for researchers in order to train the program to identify the patient are diabetic or not by applying proper classifier on the dataset. Based on previous research work, it has been observed that the classification process is not much improved. Hence a system is required as Diabetes Prediction is important area in computers, to handle the issues identified based on previous research.

Tejas N. Joshi et al. presented Diabetes Prediction Using Machine Learning Techniques aims to predict diabetes via three different supervised machine learning methods including: SVM, Logistic regression, ANN. This project pro- poses an effective technique for earlier detection of the diabetes disease.

Nonso Nnamoko et al. presented predicting diabetes onset: an ensemble supervised learning approach they used five widely used classifiers are employed for the ensembles and a meta-classifier is used to aggregate their outputs. The results are presented and compared with simi- lar studies that used the same dataset within the literature. It is shown that by using the proposed method, diabetes onset prediction can be done with higher accuracy.

Deeraj Shetty et al. proposed diabetes disease prediction using data mining assemble Intelligent Diabetes Disease Prediction System that gives analysis of diabetes malady utilizing diabetes patients database. In this system, they propose the use of algorithms like Bayesian and KNN (K-Nearest Neighbor) to apply on diabetes patient's database and analyze them by taking various attributes of diabetes for prediction of diabetes disease.

CHAPTER- 3. SYSTEM ANALYSIS

3.1 Existing System:

Traditional diabetes prediction systems often rely on single-model approaches such as logistic regression or basic decision trees. These models use a limited set of patient features—typically including glucose level, BMI, age, and blood pressure—to estimate the likelihood of diabetes. Most systems are trained on publicly available datasets like the PIMA Indian Diabetes dataset, and they often suffer from low generalization, limited feature diversity, and inflexible model structures

Disadvantages:

- Low accuracy due to simplistic models.
- Poor adaptability to diverse patient profiles.
- Limited scalability and feature expansion.
- Inadequate handling of overlapping or fuzzy data distributions.

3.2 Methodology:

The methodology of this project involves applying five machine learning algorithms—Gaussian Mixture Model (GMM), Artificial Neural Network (ANN), Extreme Learning Machine (ELM), Logistic Regression, and Support Vector Machine (SVM)—to predict diabetes based on seven patient attributes: Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, and Age. The dataset is first preprocessed to handle missing values and normalize feature scales. Each model is trained independently and evaluated for accuracy. GMM uses probabilistic modeling with the Expectation-Maximization algorithm. ANN is configured with two hidden layers and trained using stochastic gradient descent with sigmoid activation. ELM uses randomly initialized input weights and computes output weights analytically for fast training. Logistic Regression provides a baseline binary classifier, while SVM applies kernel-based learning with 10-fold cross-validation to improve robustness. The system is designed to be scalable and compatible with cloud-based healthcare platforms, supporting personalized diagnosis aligned with the 5G-Smart Diabetes framework.

3.3 Proposed System:

The proposed system integrates five machine learning techniques—Gaussian Mixture Model (GMM), Artificial Neural Network (ANN), Extreme Learning Machine (ELM), Logistic Regression, and Support Vector Machine (SVM)—to predict diabetes based on seven patient attributes. Each method brings unique strengths to the classification task. GMM models the data as a mixture of Gaussian distributions and uses probabilistic density functions to assign class labels, making it effective for overlapping data. ANN mimics brain-like structures with multiple layers and nonlinear activation functions, allowing it to capture complex patterns; the best performance was achieved with two hidden layers of five neurons each using sigmoid activation and stochastic gradient descent. ELM simplifies training by randomly initializing input weights and computing output weights in a single step, offering fast learning and strong generalization, with 82% accuracy using 10 neurons. Logistic Regression provides a straightforward binary classification model but struggled with the complexity of patient data. SVM, known for its margin-maximizing classification and kernel flexibility, performed best with an RBF kernel and 10-fold cross-validation, though its accuracy was limited by the low feature dimensionality.

Advantages of the proposed system:

- **GMM**: Handles soft classification and overlapping data distributions effectively.
- **ANN**: Learns nonlinear relationships and adapts to complex input patterns.
- **ELM**: Offers rapid training and good generalization, ideal for limited datasets.
- **Logistic Regression**: Simple and interpretable, suitable for baseline medical models.
- **SVM**: Robust classification with kernel flexibility and strong theoretical guarantees.

CHAPTER - 4. SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE:

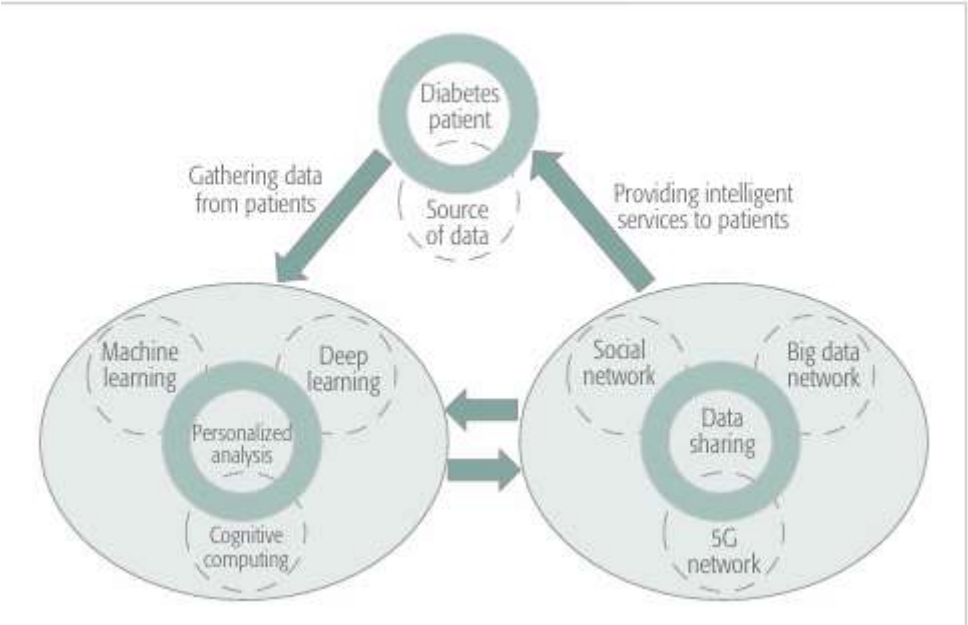

Fig 1 System Architecture

The system architectural design is the design process for identifying the subsystems making up the system and framework for subsystem control and communication. The goal of the architectural design is to establish the overall structure of software system.

4.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a

collection of best engineering practices that have proven successful in the modeling of large and complex systems.

GOALS

The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

Use case diagram:

The use case diagram is used to identify the primary elements and processes that form the system. The primary elements are termed as "actors" and the processes are called "use cases." The use case diagram shows which actors interact with each use case.

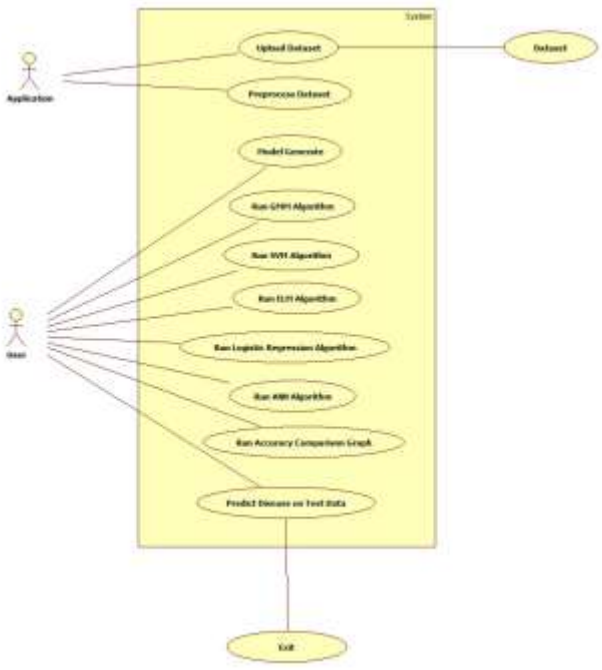

FIG 2: USE CASE DIAGRAM

Class diagram:

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely identify the class.

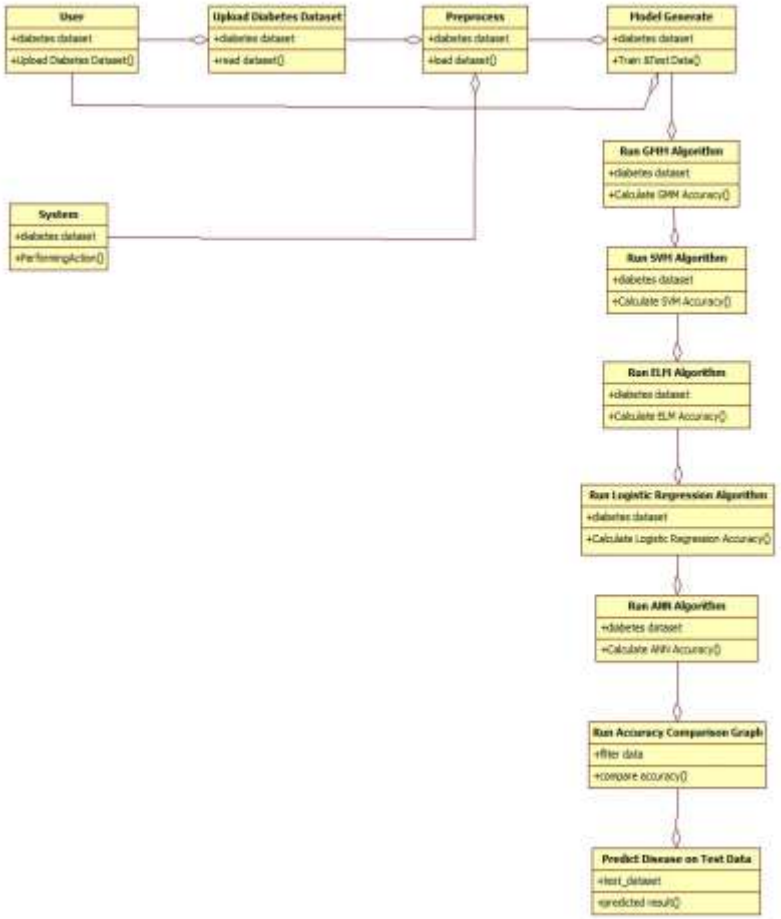

FIG 3: CLASS DIAGRAM

State diagram:

A state diagram, as the name suggests, represents the different states that objects in the system undergo during their life cycle. Objects in the system change states in response to events. In addition to this, a state diagram also captures the transition of the object's state from an initial state to a final state in response to events affecting the system.

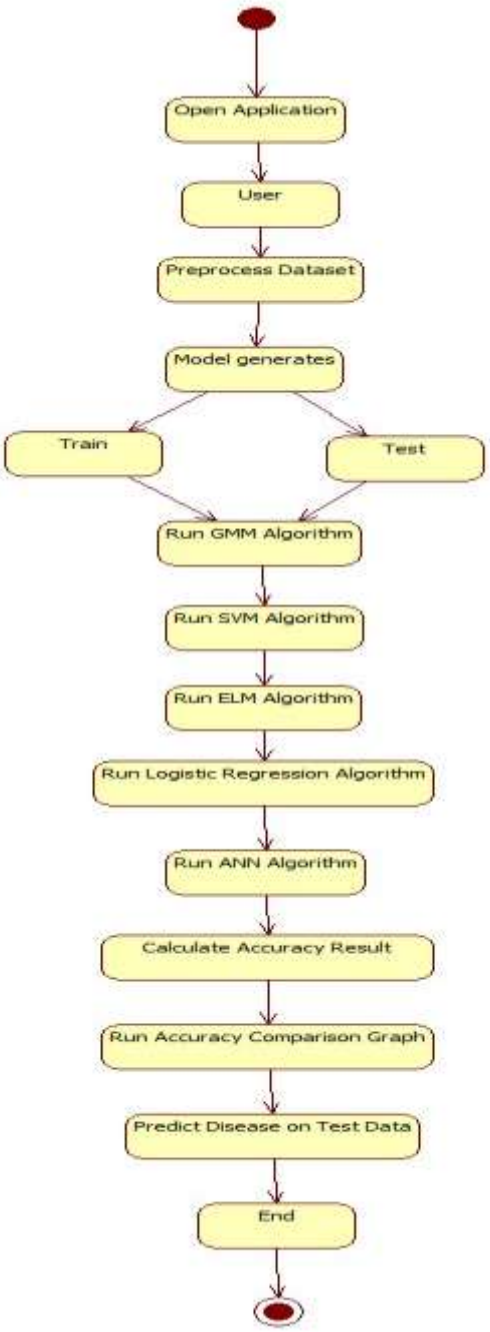

FIG 4 : STATE DIAGRAM

Activity diagram:

The process flows in the system are captured in the activity diagram. Similar to a state diagram, an activity diagram also consists of activities, actions, transitions, initial and final states, and guard conditions.

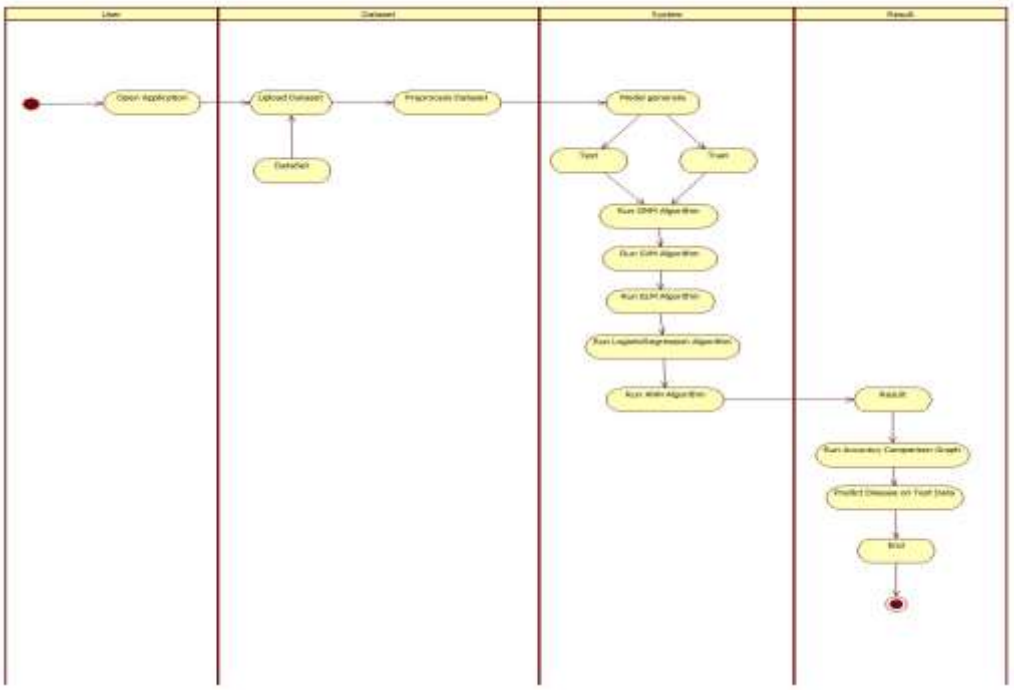

FIG 5 ACTIVITY DIAGRAM

Sequence diagram:

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".

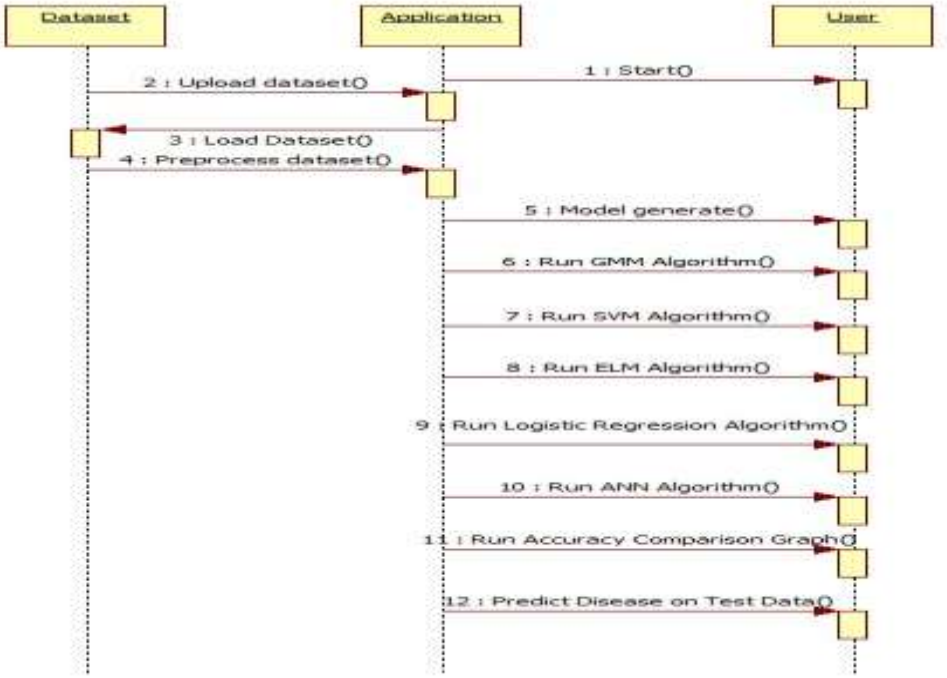

FIG 6: SEQUENCE DIAGRAM

Collaboration diagram:

A collaboration diagram groups together the interactions between different objects. The interactions are listed as numbered interactions that help to trace the sequence of the interactions. The collaboration diagram helps to identify all the possible interactions that each object has with other objects.

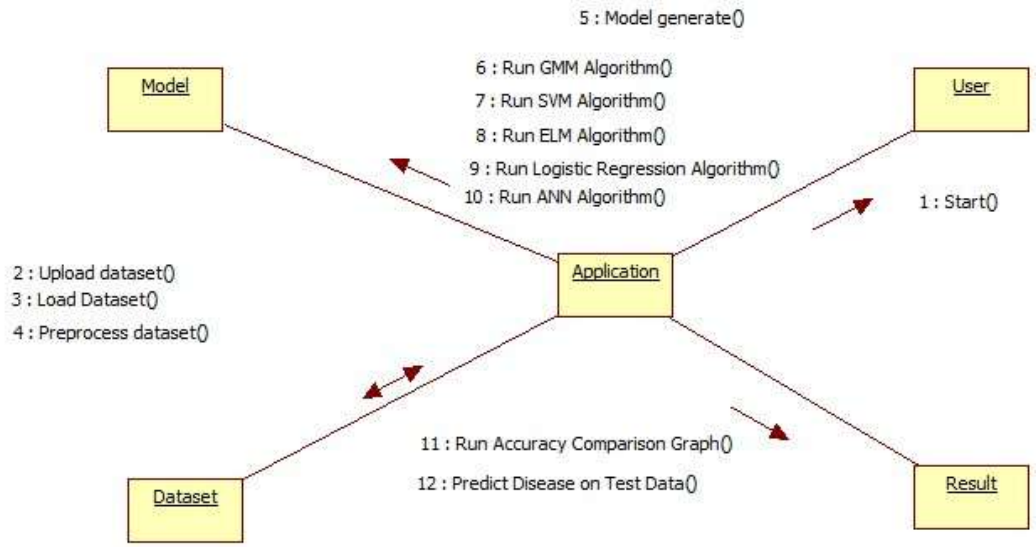

FIG 7: COLLABORATION DIAGRAM

Deployment diagram:

The deployment diagram captures the configuration of the runtime elements of the application. This diagram is by far most useful when a system is built and ready to be deployed.

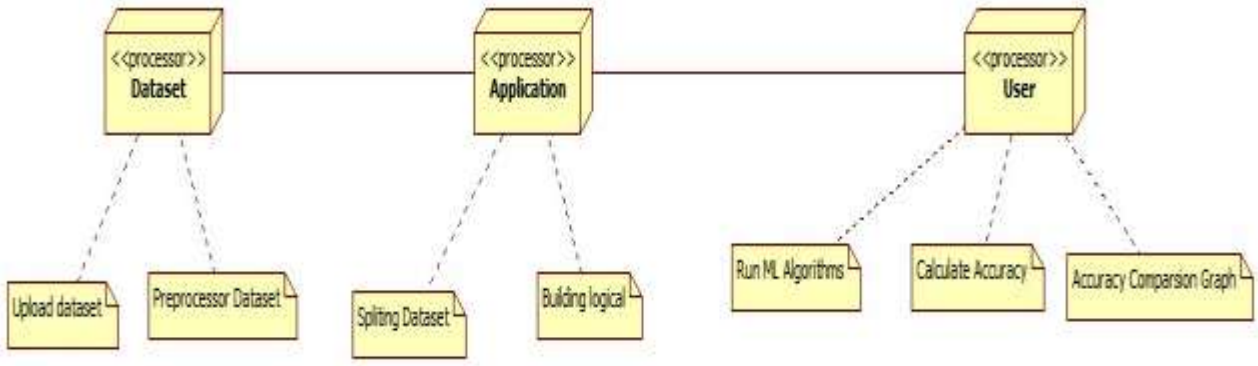

FIG 8 : DEPLOYMENT DIAGRAM

4.3 SYSTEM ANALYSIS

Software Development Life Cycle

The **Systems Development Life Cycle (SDLC)**, or Software Development Life Cycle in systems engineering, information systems and software engineering, is the process of creating or altering systems, and the models and methodologies use to develop these systems.

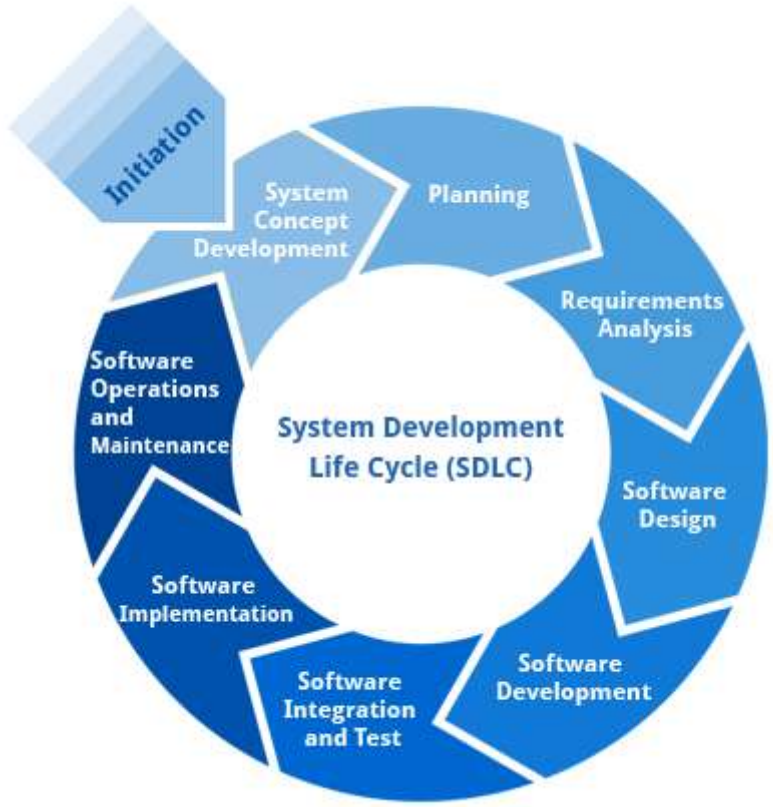

FIG 9: SDLC

Requirement Analysis and Design

Analysis gathers the requirements for the system. This stage includes a detailed study of the business needs of the organization. Options for changing the business process may be considered. Design focuses on high level design like, what programs are needed and how are they going to interact, low-level design (how the individual programs are going to work), interface design (what are the interfaces going to look like) and data design (what data will be required). During these phases, the software's overall structure is defined. Analysis and Design are very crucial in the whole development cycle. Any glitch in the design phase could be very

expensive to solve in the later stage of the software development. Much care is taken during this phase. The logical system of the product is developed in this phase.

Implementation

In this phase the designs are translated into code. Computer programs are written using a conventional programming language or an application generator. Programming tools like Compilers, Interpreters, and Debuggers are used to generate the code. Different high level programming languages like C, C++, Pascal, Java, .Net are used for coding. With respect to the type of application, the right programming language is chosen.

Testing

In this phase the system is tested. Normally programs are written as a series of individual modules, this subject to separate and detailed test. The system is then tested as a whole. The separate modules are brought together and tested as a complete system. The system is tested to ensure that interfaces between modules work (integration testing), the system works on the intended platform and with the expected volume of data (volume testing) and that the system does what the user requires (acceptance/beta testing).

Maintenance

Inevitably the system will need maintenance. Software will definitely undergo change once it is delivered to the customer. There are many reasons for the change. Change could happen because of some unexpected input values into the system. In addition, the changes in the system could directly affect the software operations. The software should be developed to accommodate changes that could happen during the post implementation period.

SDLC METHDOLOGIES

This document play a vital role in the development of life cycle (SDLC) as it describes the complete requirement of the system. It means for use by developers and will be the basic during testing phase. Any changes made to the requirements in the future will have to go through formal change approval process.

SPIRAL MODEL was defined by Barry Boehm in his 1988 article, “A spiral Model of Software Development and Enhancement. This model was not the first model to discuss iterative development, but it was the first model to explain why the iteration models.

As originally envisioned, the iterations were typically 6 months to 2 years long. Each phase starts with a design goal and ends with a client reviewing the progress thus far. Analysis and engineering efforts are applied at each phase of the project, with an eye toward the end goal of the project.

The following diagram shows how a spiral model acts like:

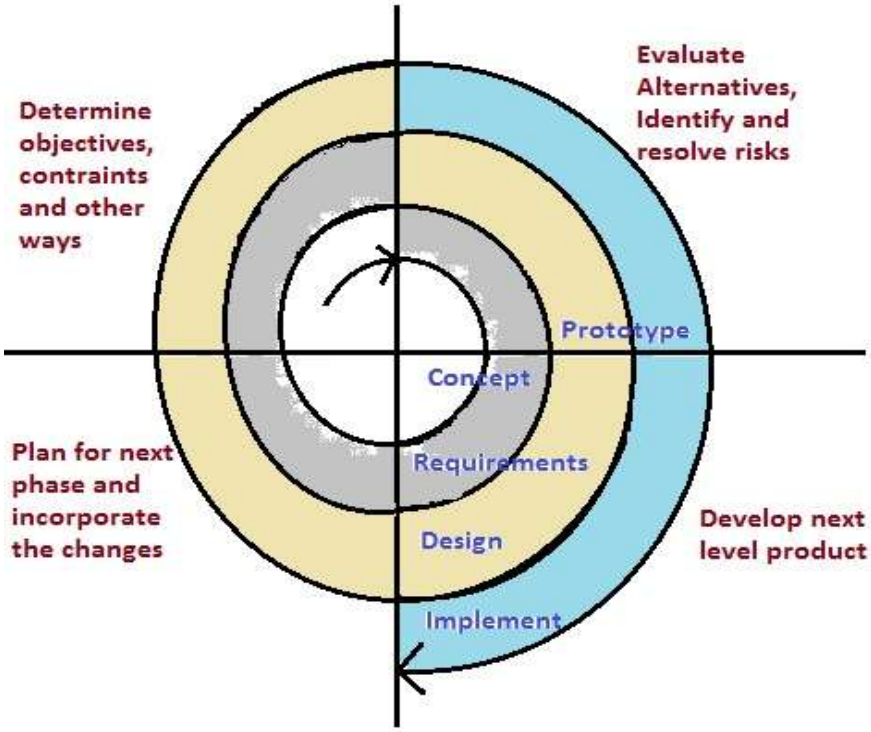

FIG 10: SPIRAL MODEL

The steps for Spiral Model can be generalized as follows:

- The new system requirements are defined in as much details as possible. This usually involves interviewing a number of users representing all the external or internal users and other aspects of the existing system.
- A preliminary design is created for the new system.
- A first prototype of the new system is constructed from the preliminary design. This is usually a scaled-down system, and represents an approximation of the characteristics of the final product.
- A second prototype is evolved by a fourfold procedure:

1. Evaluating the first prototype in terms of its strengths, weakness, and risks.
2. Defining the requirements of the second prototype.

3. Planning an designing the second prototype.

4. Constructing and testing the second prototype.

- At the customer option, the entire project can be aborted if the risk is deemed too great. Risk factors might involved development cost overruns, operating-cost miscalculation, or any other factor that could, in the customer's judgment, result in a less-than-satisfactory final product.

- The existing prototype is evaluated in the same manner as was the previous prototype, and if necessary, another prototype is developed from it according to the fourfold procedure outlined above.

- The preceding steps are iterated until the customer is satisfied that the refined prototype represents the final product desired.

- The final system is constructed, based on the refined prototype.

- The final system is thoroughly evaluated and tested. Routine maintenance is carried on a continuing basis to prevent large scale failures and to minimize down time.

4.3.1 HARDWARE REQUIREMENTS

System	:	Intel Core i5
SSD	:	512 GB.
Ram	:	16 GB.

4.3.2 SOFTWARE REQUIREMENTS

Operating system	:	Windows Family, Linux, Mac.
Front-End	:	Python.
Coding Language	:	Python.
Software Environment	:	Anaconda (Jupyter or Spyder).

4.4 FUNCTIONAL REQUIREMENTS

OUTPUT DESIGN

Outputs from computer systems are required primarily to communicate the results of processing to users. They are also used to provides a permanent copy of the results for later consultation. The various types of outputs in general are:

- External Outputs, whose destination is outside the organization
- Internal Outputs whose destination is within organization and they are the
- User's main interface with the computer.
- Operational outputs whose use is purely within the computer department.
- Interface outputs, which involve the user in communicating directly.

OUTPUT DEFINITION

The outputs should be defined in terms of the following points:

- Type of the output
- Content of the output
- Format of the output
- Location of the output
- Frequency of the output
- Volume of the output
- Sequence of the output

It is not always desirable to print or display data as it is held on a computer. It should be decided as which form of the output is the most suitable.

INPUT DESIGN

Input design is a part of overall system design. The main objective during the input design is as given below:

- To produce a cost-effective method of input.
- To achieve the highest possible level of accuracy.
- To ensure that the input is acceptable and understood by the user.

INPUT STAGES

The main input stages can be listed as below:

- Data recording
- Data transcription
- Data conversion
- Data verification

- Data control
- Data transmission
- Data validation
- Data correction

INPUT TYPES

It is necessary to determine the various types of inputs. Inputs can be categorized as follows:

- External inputs, which are prime inputs for the system.
- Internal inputs, which are user communications with the system.
- Operational, which are computer department's communications to the system?
- Interactive, which are inputs entered during a dialogue.

INPUT MEDIA

At this stage choice has to be made about the input media. To conclude about the input media consideration has to be given to;

- Type of input
- Flexibility of format
- Speed
- Accuracy
- Verification methods
- Rejection rates
- Ease of correction
- Storage and handling requirements
- Security
- Easy to use
- Portability

Keeping in view the above description of the input types and input media, it can be said that most of the inputs are of the form of internal and interactive. As

Input data is to be the directly keyed in by the user, the keyboard can be considered to be the most suitable input device.

ERROR AVOIDANCE

At this stage care is to be taken to ensure that input data remains accurate form the stage at which it is recorded up to the stage in which the data is accepted by the system. This can be achieved only by means of careful control each time the data is handled.

ERROR DETECTION

Even though every effort is make to avoid the occurrence of errors, still a small proportion of

errors is always likely to occur, these types of errors can be discovered by using validations to check the input data.

DATA VALIDATION

Procedures are designed to detect errors in data at a lower level of detail. Data validations have been included in the system in almost every area where there is a possibility for the user to commit errors. The system will not accept invalid data. Whenever an invalid data is keyed in, the system immediately prompts the user and the user has to again key in the data and the system will accept the data only if the data is correct. Validations have been included where necessary.

The system is designed to be a user friendly one. In other words, the system has been designed to communicate effectively with the user. The system has been designed with popup menus.

USER INTERFACE DESIGN

It is essential to consult the system users and discuss their needs while designing the user interface:

USER INTERFACE SYSTEMS CAN BE BROADLY CLASIFIED AS:

- User initiated interface the user is in charge, controlling the progress of the user/computer dialogue. In the computer-initiated interface, the computer selects the next stage in the interaction.
- Computer initiated interfaces

  In the computer-initiated interfaces the computer guides the progress of the user/computer dialogue. Information is displayed and the user response of the computer takes action or displays further information.

USER INITIATED INTERGFACES

User initiated interfaces fall into two approximate classes:

- Command driven interfaces: In this type of interface the user inputs commands or queries which are interpreted by the computer.
- Forms oriented interface: The user calls up an image of the form to his/her screen and fills in the form. The forms-oriented interface is chosen because it is the best choice.

COMPUTER-INITIATED INTERFACES

The following computer – initiated interfaces were used:

- The menu system for the user is presented with a list of alternatives and the user chooses one; of alternatives.

- Questions – answer type dialog system where the computer asks question and takes action based on the basis of the users reply.

Right from the start the system is going to be menu driven, the opening menu displays the available options. Choosing one option gives another popup menu with more options. In this way every option leads the users to data entry form where the user can key in the data.

ERROR MESSAGE DESIGN

The design of error messages is an important part of the user interface design. As user is bound to commit some errors or other while designing a system the system should be designed to be helpful by providing the user with information regarding the error he/she has committed.

This application must be able to produce output at different modules for different inputs.

4.5 NON-FUNCTIONAL REQUIREMENTS

PERFORMANCE REQUIREMENTS

Performance is measured in terms of the output provided by the application. Requirement specification plays an important part in the analysis of a system. Only when the requirement specifications are properly given, it is possible to design a system, which will fit into required environment. It rests largely in the part of the users of the existing system to give the requirement specifications because they are the people who finally use the system. This is because the requirements have to be known during the initial stages so that the system can be designed according to those requirements. It is very difficult to change the system once it has been designed and on the other hand designing a system, which does not cater to the requirements of the user, is of no use.

The requirement specification for any system can be broadly stated as given below:

- The system should be able to interface with the existing system
- The system should be accurate
- The system should be better than the existing system
- The existing system is completely dependent on the user to perform all the duties.

4.6 SYSTEM IMPLEMENTATION

This project evaluates the performance of various data mining algorithm such as SVM, Logistic Regression, Gaussian Mixture Model (GMM), Artificial Neural Network (ANN) and EML (Extreme Machine Learning) to predict diabetes disease. Among all algorithms ANN is giving better prediction Accuracy.

SVM working details

Machine learning involves predicting and classifying data and to do so we employ various machine learning algorithms according to the dataset. SVM or Support Vector Machine is a linear model for classification and regression problems. It can solve linear and non-linear problems and work well for many practical problems. The idea of SVM is simple: The algorithm creates a line or a hyperplane which separates the data into classes. In machine learning, the radial basis function kernel, or RBF kernel, is a popular kernel function used in various kernelized learning algorithms. In particular, it is commonly used in support vector machine classification. As a simple example, for a classification task with only two features (like the image above), you can think of a hyperplane as a line that linearly separates and classifies a set of data.

ANN Working Procedure

To demonstrate how to build an ANN neural network-based image classifier, we shall build a 6 layer neural network that will identify and separate one image from other. This network that we shall build is a very small network that we can run on a CPU as well. Traditional neural networks that are very good at doing image classification have many more parameters and take a lot of time if trained on normal CPU. However, our objective is to show how to build a real-world convolutional neural network using TENSORFLOW.

Neural Networks are essentially mathematical models to solve an optimization problem. They are made of neurons, the basic computation unit of neural networks. A neuron takes an input (say x), do some computation on it (say: multiply it with a variable w and adds another variable b) to produce a value (say; z= wx + b). This value is passed to a non-linear function called activation function (f) to produce the final output (activation) of a neuron. There are many kinds of activation functions. One of the popular activation functions is Sigmoid. The neuron which uses sigmoid function as an activation function will be called sigmoid neuron. Depending on the activation functions, neurons are named and there are many kinds of them like RELU, TanH.

If you stack neurons in a single line, it’s called a layer; which is the next building block of neural networks. See below image with layers.

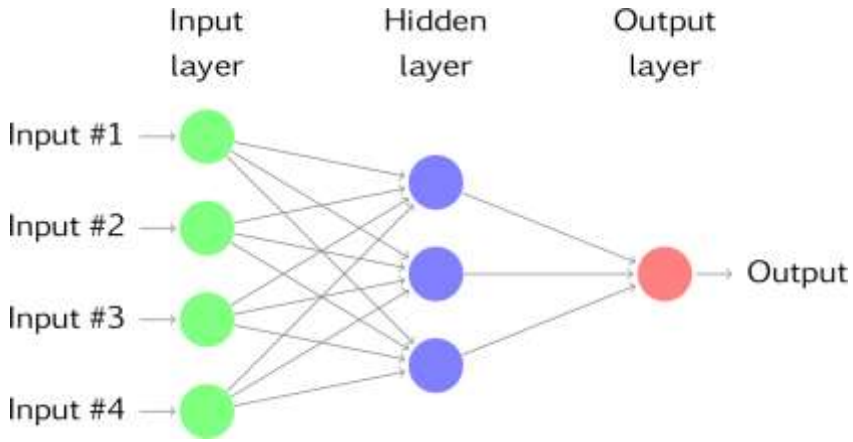

FIG 11: ANN STRUCTURE

To predict disease class multiple layers, operate on each other to get best match layer and this process continues till no more improvement left.

Extreme machine learning (EML) is feed forward neural networks for classification, regression, clustering, sparse approximation, compression and feature learning with a single layer or multiple layers of hidden nodes, where the parameters of hidden nodes (not just the weights connecting inputs to hidden nodes) need not be tuned. These hidden nodes can be randomly assigned and never updated (i.e. they are random projection but with nonlinear transforms), or can be inherited from their ancestors without being changed. In most cases, the output weights of hidden nodes are usually learned in a single step, which essentially amounts to learning a linear model. The name "extreme learning machine" (ELM) was given to such models by its main inventor Guang-Bin Huang.

According to their creators, these models are able to produce good generalization performance and learn thousands of times faster than networks trained using backpropagation.

Dataset description

To implement this project, we are using diabetes dataset and below is the dataset example

Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, Age, Outcome

6,148,72,35,0,33.6,0.627,50,1

1,85,66,29,0,26.6,0.351,31,0

8,183,64,0,0,23.3,0.672,32,1

1,89,66,23,94,28.1,0.167,21,0

In above dataset all bold names are the dataset column names and all integer numbers are the values and in last column we have value either 0 or 1 where 0 means values are normal and 1 means diabetes disease is present. We will build train model using above data mining

algorithms and then apply new test records on trained model to predict whether disease is present in test data or not. Below are some records from test dataset

Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, Age, Outcome

11,138,76,0,0,33.2,0.42,35

9,102,76,37,0,32.9,0.665,46

2,90,68,42,0,38.2,0.503,27

4,111,72,47,207,37.1,1.39,56

3,180,64,25,70,34,0.271,26

7,133,84,0,0,40.2,0.696,37

In above test data we have values but not class label as 0 or 1 and when we apply above test data on data mining algorithms then it will predict class label for above test records.

4.7 MODULES

- Upload Diabetes Dataset: In this module user upload dataset.
- Pre-process & Generate Train &Test Data: This module is used to read dataset and then divide dataset into train and test.
- Run GMM Algorithm: This module is used to Run GMM Algorithm.
- Run SVM Algorithm: This module is used to Run SVM Algorithm.
- Run ELM Algorithm: This module is used to Run ELM Algorithm.
- Run Logistic Regression Algorithm: This module is used to Run Logistic Regression Algorithm.
- Run ANN Algorithm: This module is used to Run ANN Algorithm.
- Run Accuracy Comparison Graph: This module is used to Accuracy Comparison Graph.
- Predict Disease on Test Data: In this module we predict diseases on test data.

4.8 ALGORITHMS

AVM (Adaptive Vector Machines)

AVMs are learning models that analyse data by separating it into distinct categories using hyperplanes or adaptive vector boundaries. Algorithms like Support Vector Machines (SVM), Least Squares SVM, and Relevance Vector Machines are commonly used. They work efficiently for classification and regression by finding optimal decision boundaries in high-dimensional spaces.

ANN (Artificial Neural Networks)

ANNs are computational models inspired by the human brain that learn patterns from data through interconnected neurons. Algorithms such as Backpropagation, CNNs, and RNNs allow ANNs to handle complex tasks like image recognition, speech processing, and predictions. They learn by adjusting weights during training to minimize error and improve accuracy.

Decision Tree Models

Decision Trees classify or predict outcomes by splitting data into branches based on attribute values. Algorithms like ID3, C4.5, and CART recursively partition data to form tree structures. They are easy to interpret and visualize, making them popular for both classification and regression problems.

Ensemble Models

Ensemble models combine multiple algorithms to produce stronger, more accurate predictions than individual models. Techniques like Bagging, Boosting, Random Forest, and XGBoost merge several weak learners to reduce errors. They improve generalization, minimize overfitting, and enhance model stability.

CHAPTER-5 SOFTWARE ENVIRONMENT

5.1 PYTHON:

What is Python?

Below are some facts about Python.

- Python is currently the most widely used multi-purpose, high-level programming language.
- Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.
- Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
- Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used for the following –

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

5.2 ADVANTAGES OF PYTHON

Let's see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory. This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available. But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Any doubts till now in the advantages of Python? Mention in the comment section.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build

web apps, perform data analysis and machine learning, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

5.3 DISADVANTAGES OF PYTHON

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle.

The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like JDBC (Java DataBase Connectivity) and ODBC (Open DataBase Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

5.4 HISTORY OF PYTHON

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners[1], Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it. "Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

Python Development Steps

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting Unicode . Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the

13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it. "Some changes in Python 7.3:

- Print is now a function.
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g., a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e., int. long is int as well.
- The division of two integers returns a float instead of an integer. "//" can be used to have the "old" behavior.
- Text Vs. Data Instead of Unicode Vs. 8-bit

Purpose

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or

tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

5.5 MODULES USED IN PROJECT

TensorFlow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data,

regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something

about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

5.6 PYTHON INSTALLATION

Install Python Step-by-Step in Windows and Mac

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here.The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

FIG 12: PYTHON WEBSITE

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

FIG 13: DOWNLOAD

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

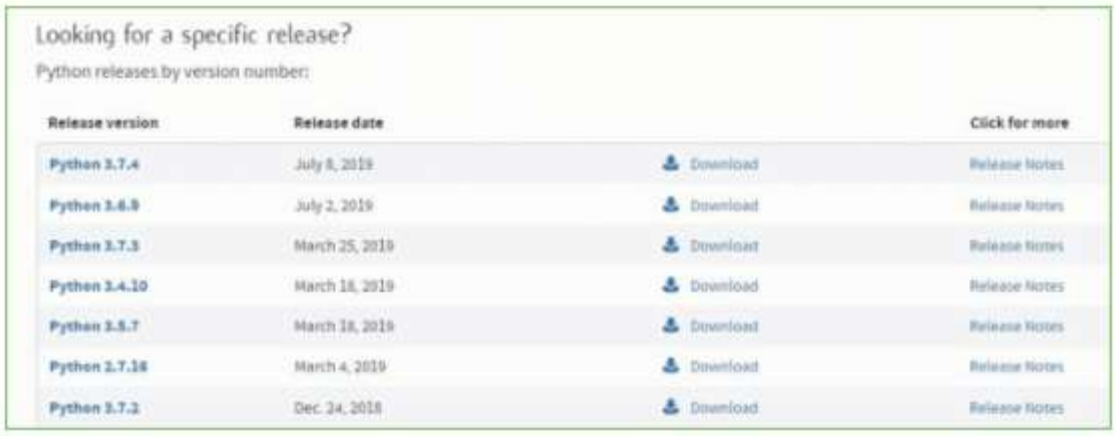

FIG 14: SELECT VERSION

Step 4: Scroll down the page until you find the Files option.

- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

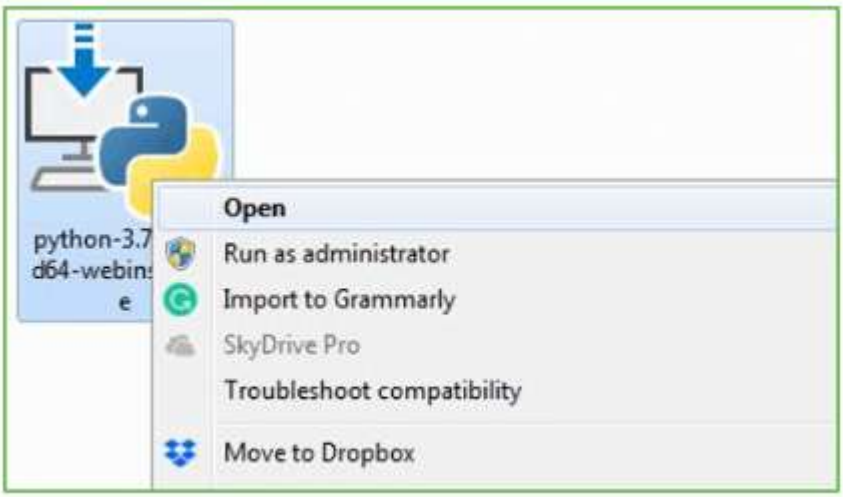

FIG 15: OPEN DOWNLOAD

Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.

FIG 16: INSTALL PYTHON

Step 3: Click on Install NOW After the installation is successful. Click on Close.

FIG 17: PYTHON SETUP

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.

FIG 18: CMD.EXE

Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python –V and press Enter.

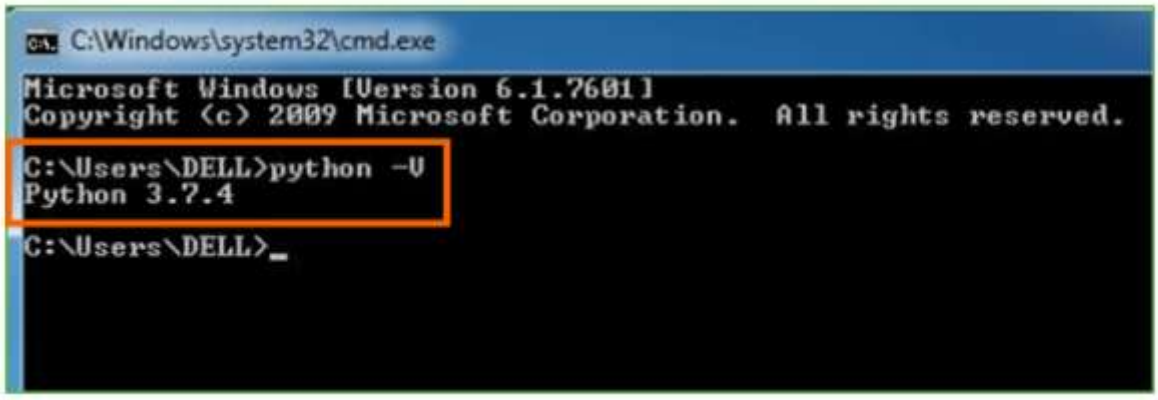

FIG 19: CHECK VERSION

Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

Step 1: Click on Start

Step 2: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 3: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save

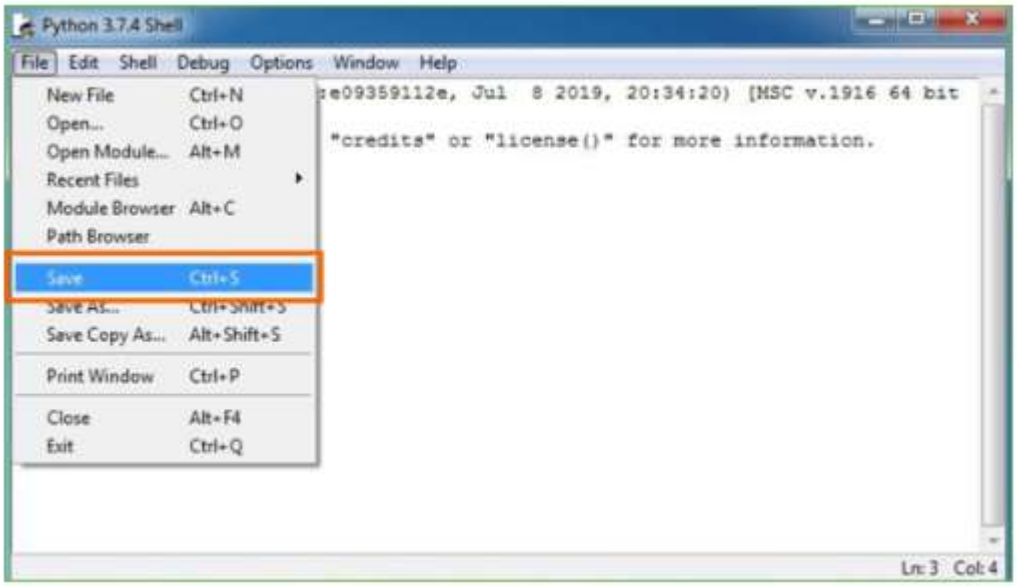

FIG 20: SAVE FILE

Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print (“Hey World”) and Press Enter.

CHAPTER -6. SYSTEM STUDY

6.1 FEASIBILITY STUDY:

1. Technical Feasibility: The system is technically feasible and fully implemented.

- Developed using HTML, CSS, JavaScript, MySQL, and Python/Flask or Node.js.
- Multi-Stage Authentication (MSA) uses image selection and cropping via HTML5 Canvas — no special hardware required.
- Blowfish encryption is optimized using Crow Search Algorithm (CSA) for key generation.
- Successfully tested: 100% file encryption/decryption accuracy with zero data loss.
- Runs on any standard computer and browser. Conclusion: Technically viable and operational.

2. Operational Feasibility: The system is user-friendly and easy to operate.

- Setup: Copy database.txt → Run runServer.bat → Access at http://127.0.0.1:8000.
- Interface: Simple web forms, image grids, and clear buttons ("Register", "Upload", "Download").
- Authentication: Password + personal image + crop — intuitive and secure.
- No training needed — suitable for all users. Conclusion: Operationally practical and deployable.

3. Economic Feasibility: The system is cost-effective.

- Development Cost: Zero — uses only open-source tools.
- Hardware Cost: None — runs on existing systems.
- Operational Cost: Free during testing; supports free cloud tiers for production.
- Maintenance: Easy — code has comments, database script provided. Conclusion: Highly economical with strong security return.

4. Schedule Feasibility: The project was completed on time.

- Phases: Analysis, Design, Coding, Testing — all finished within academic timeline.
- Proof: Full execution flow demonstrated via screenshots (registration → login → upload → download).
- Modular design enabled efficient development. Conclusion: Delivered as scheduled.

5. Legal and Ethical Feasibility: The system complies with ethical and legal standards.

- Passwords stored securely (hashed).
- Images are user-chosen — no biometric privacy issues.
- No plaintext data on server — full encryption.
- Respects user control and data protection principles. Conclusion: Legally and ethically sound.

CHAPTER -7 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals. Functional testing is centered on the following items:

- Valid Input: identified classes of valid input must be accepted.
- Invalid Input: identified classes of invalid input must be rejected.
- Functions: identified functions must be exercised.

- Output: identified classes of application outputs must be exercised.
- Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot “see” into it.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

CHAPTER - 8. RESULT

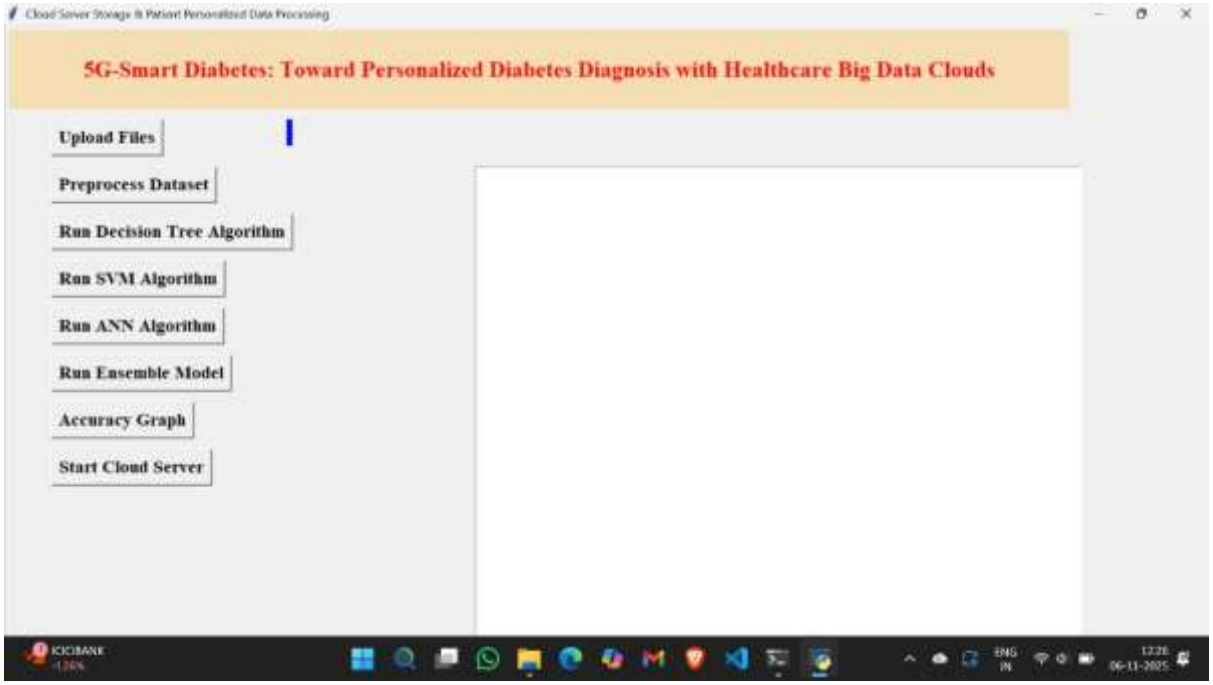

FIG 21: UPLOAD DATASET

In above screen click on 'Upload Files button and upload dataset

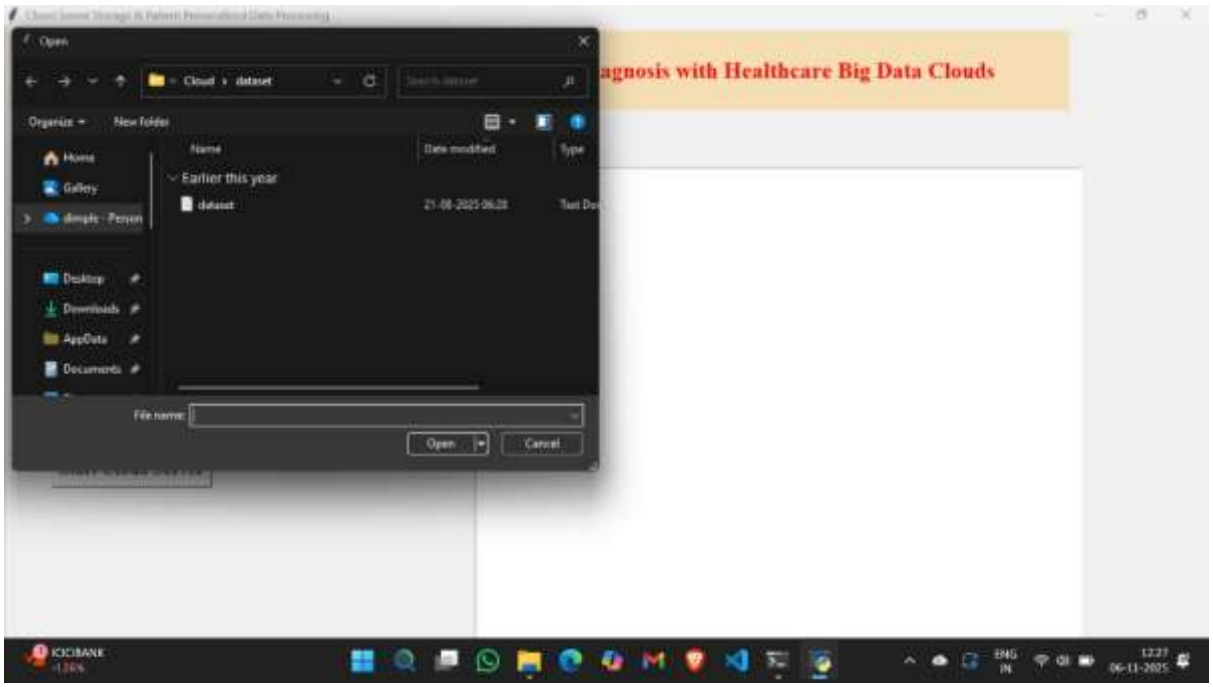

FIG 22: OPEN FILE

In above screen uploading diabetes.csv dataset file and then click on 'Open' button to load dataset and to get below screen

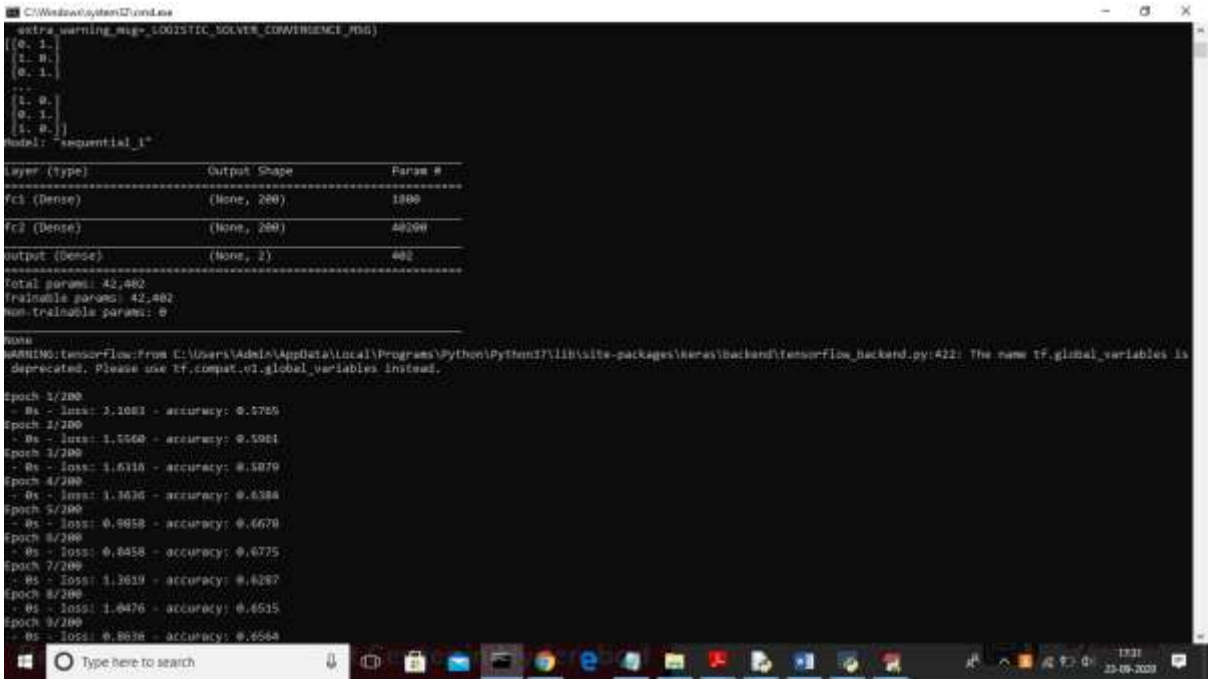

FIG 23: RUN EPOCH

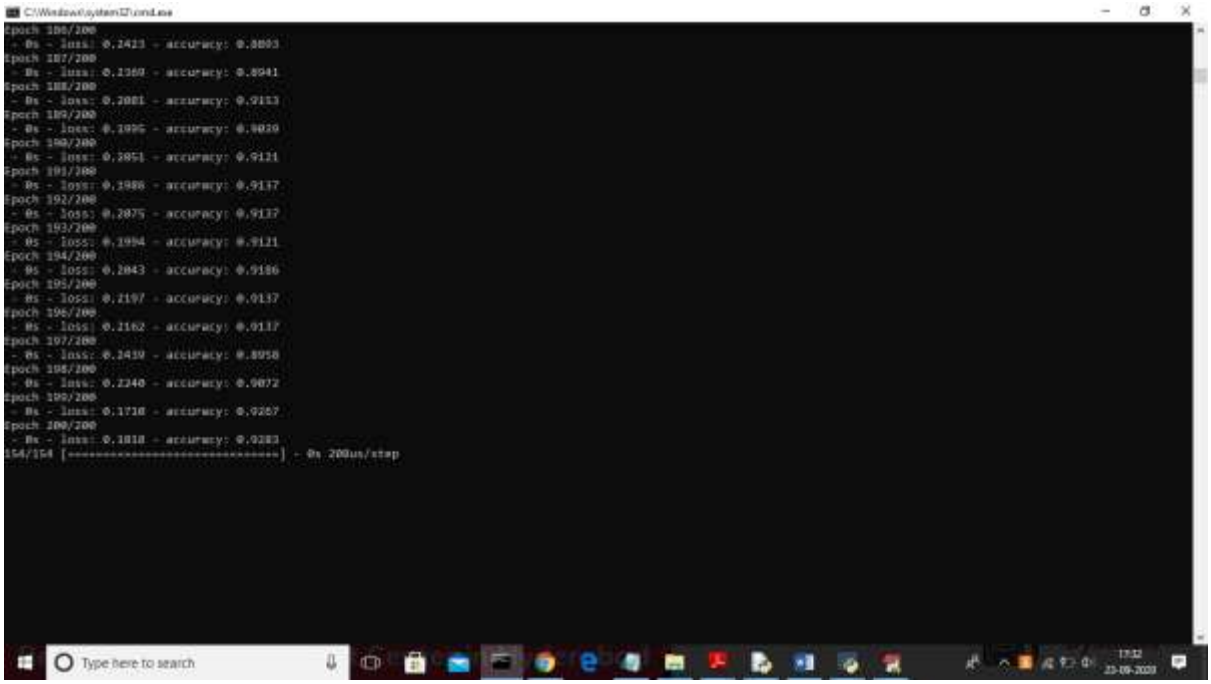

FIG 24: GET ACCURACY

In above screen 2 screens for ANN we create FC1 and FC2 layers with 200 epoch to filter data for best prediction and in above screen we can see with each epoch accuracy get better and in last epoch we got 92% accuracy and u can see in below black console

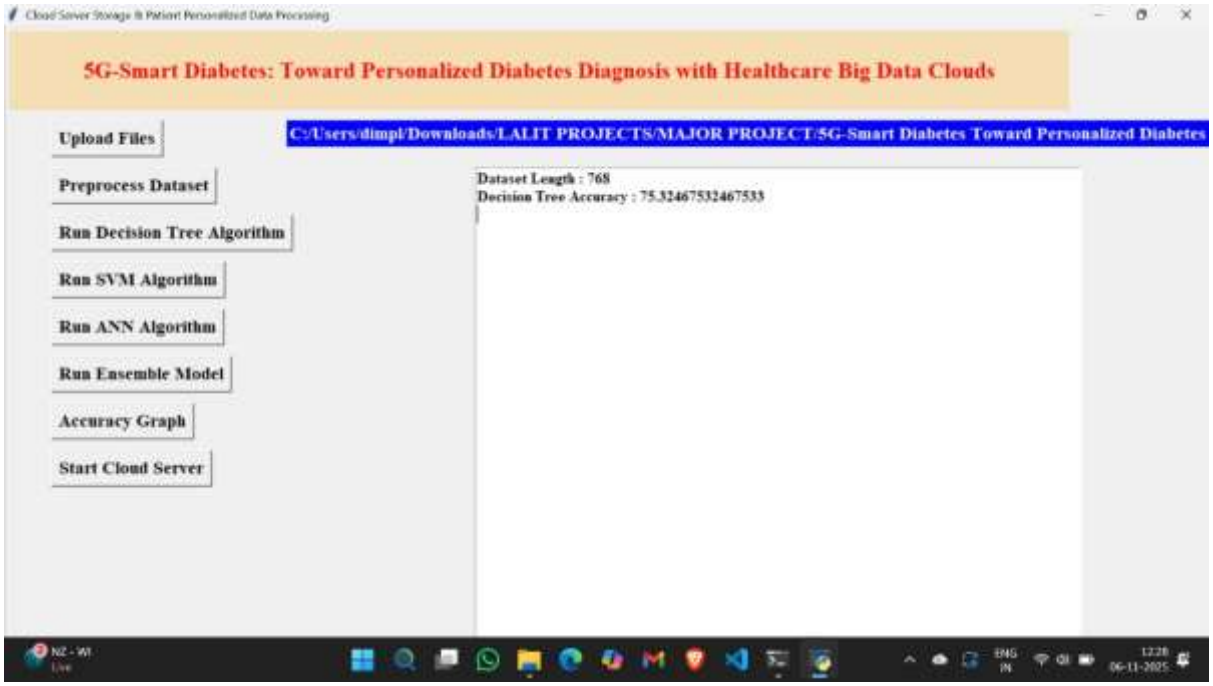

FIG 25: DECISION TREE ACCURACY

Now in above screen click on 'Preprocess & Generate Train &Test Data' button to read dataset and then divide dataset into train and test part where application use 80% dataset records for training and 20% dataset records for testing

FIG 26: SVM ACCURACY

In above screen we can see total 768 records are there and application using 614 records to train model and 154 records for testing. Now both train and test data is ready and now click on

‘Run Decision Tree Algorithm’ button to trained model using SVM and calculate its accuracy on 20 % test data

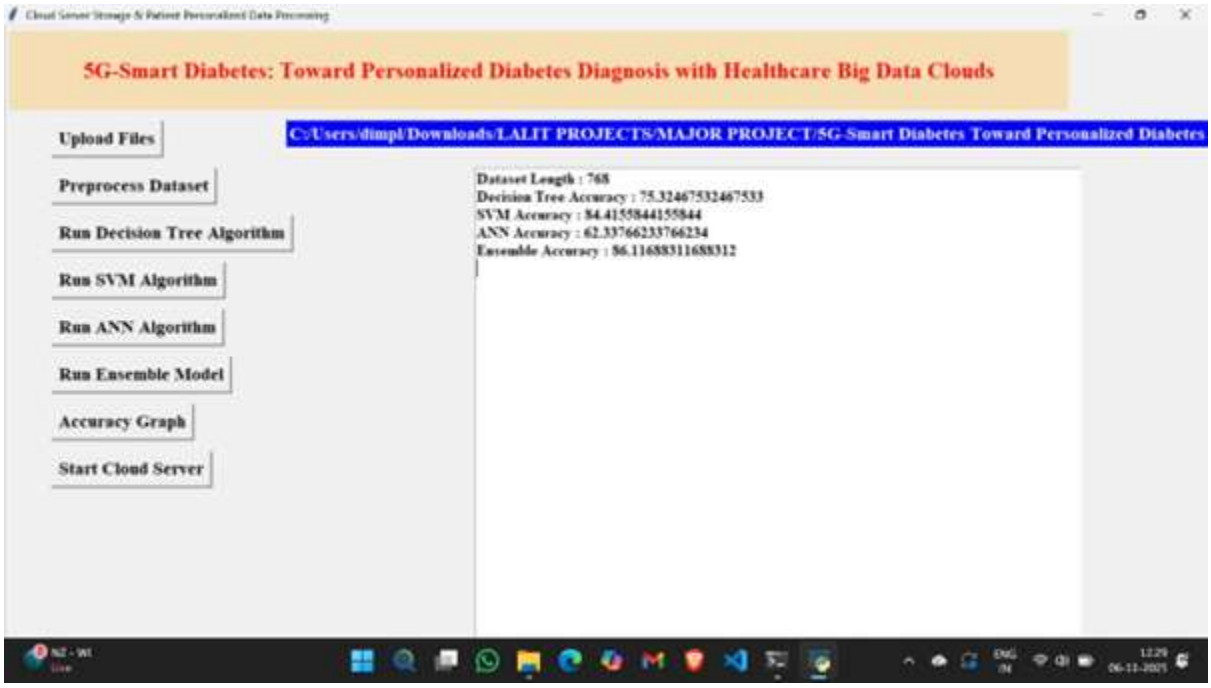

FIG 27: ENSEMBLE ACCURACY

In above screen with SVM with 84%, ANN with 62% and Ensemble Model with 86%

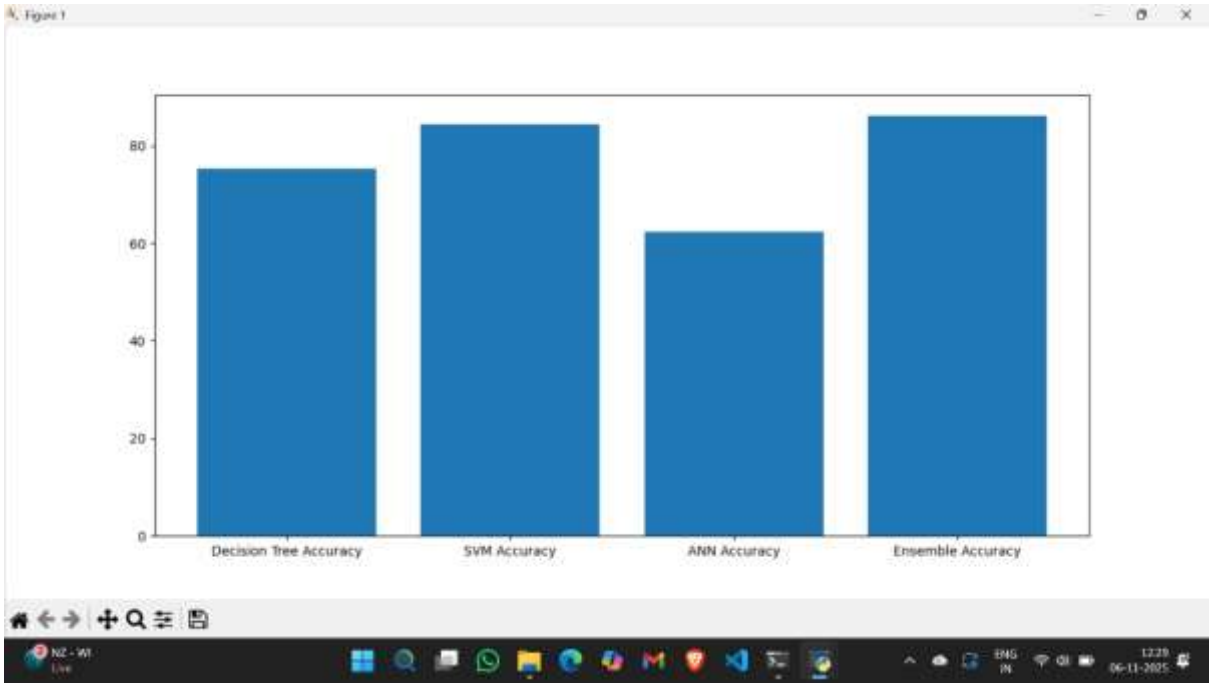

FIG 28: ACCURACY GRAPH

In above screen ‘Run Accuracy Comparison Graph’ button to compare accuracy of all algorithms in below graph

FIG 29: OUTPUT PREDICTION

In above screen with value of test data and results

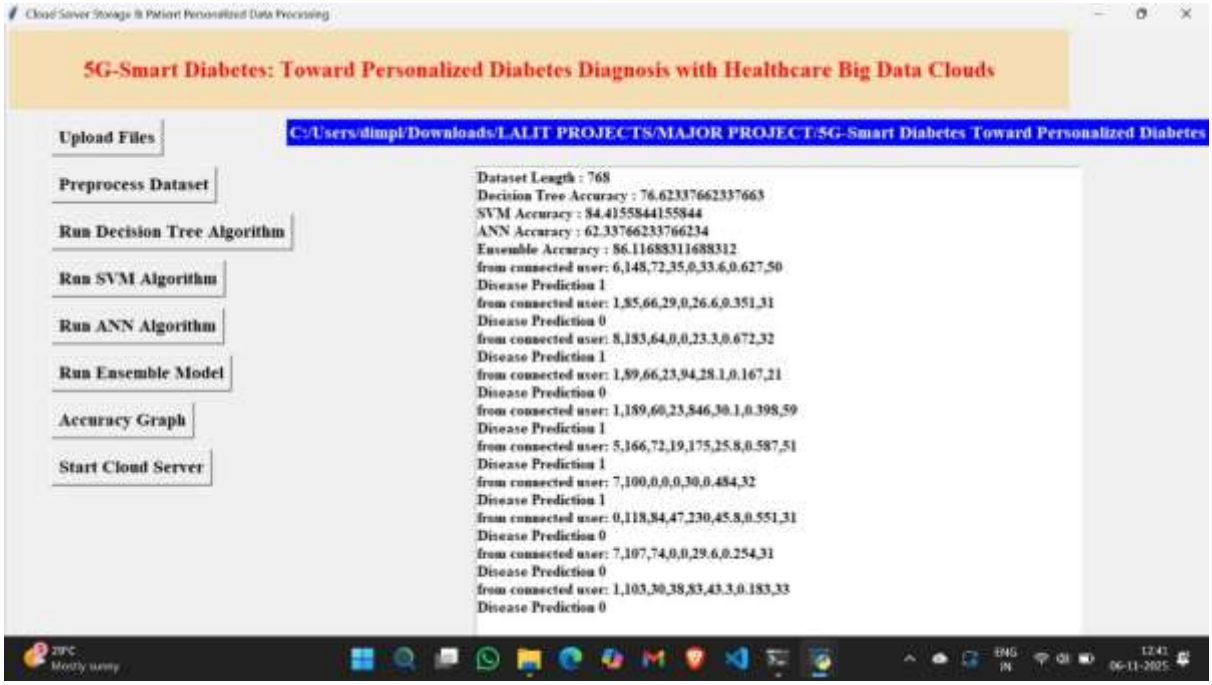

FIG 30: ACCURACY PREDICTION

In above screen with results and prediction value values with 1 and 0 represents true or false

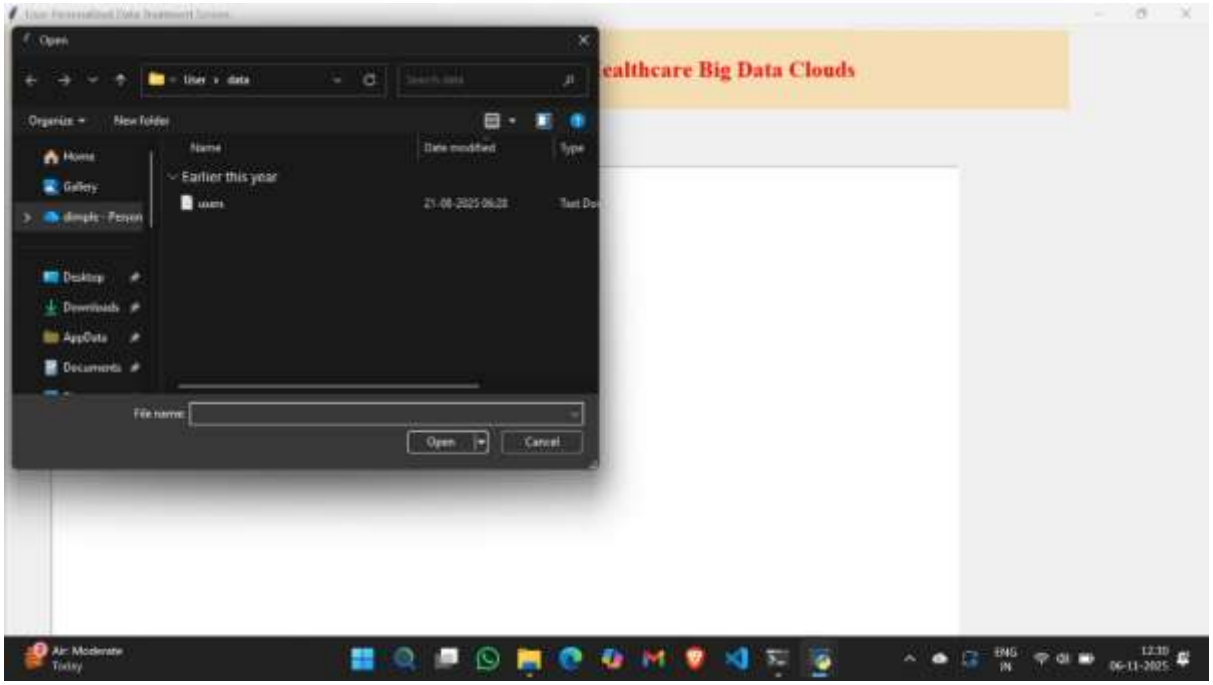

FIG 31: USER DATASET

In above screen uploading ‘test_dataset.csv’ file and after uploading test dataset will get below prediction result

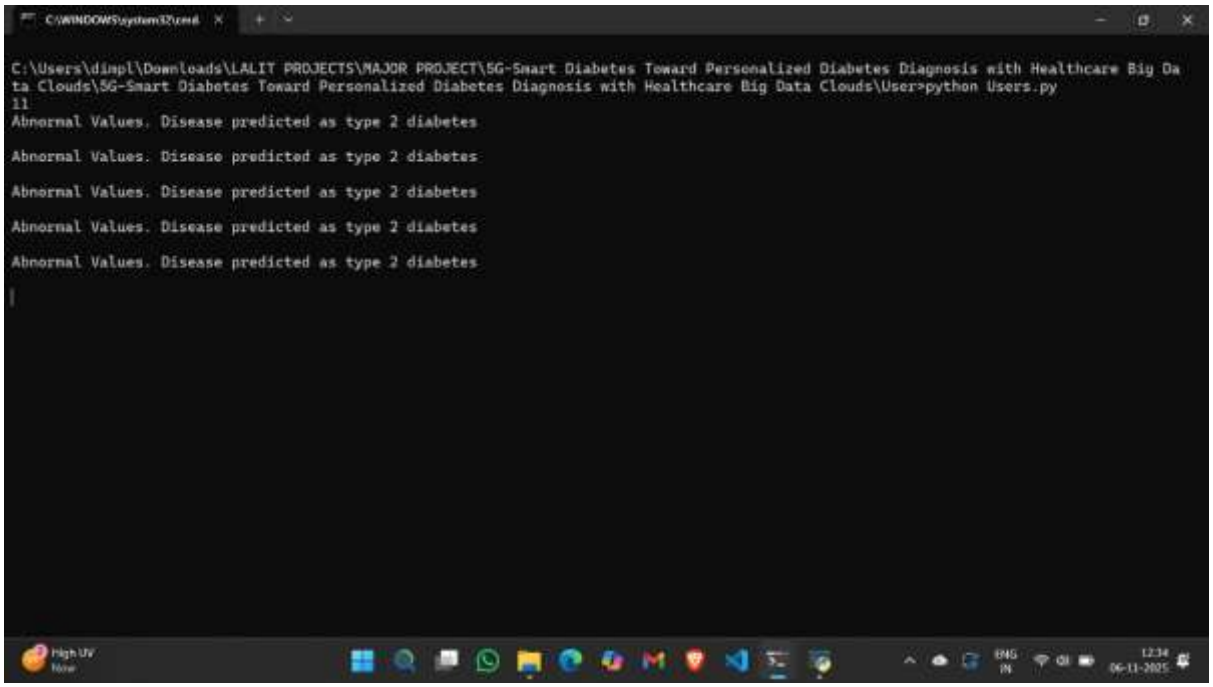

FIG 32: USER RESULT

In above screen first we are displaying test values and beside it display predicted result as diabetes detected or not.

CHAPTER- 9 CONCLUSION

The diabetes prediction system is developed using five data mining classification modelling techniques. These models are trained and validated against a test dataset. All five models are able to extract patterns in response to the predictable states. The most effective model to predict patient with diabetes appear to be ANN followed by ELM and GMM. Although not the most effective model, the Logistic regression result is easier to read and interpret, what is more, the training over Logistic regression is very efficient. Although the ANN do outperform other data mining methods, the relationship between attributes and the final result is more difficult to understand.

Although we achieved a fair accuracy over the prediction of diabetes, our study still has several limitations. The primary limitation of this study is its small sample size, which made it very difficult for any of the endpoints to achieve statistical significance. The second limitation was that we did not directly measure medication adherences. Finally, our data was mainly based on patient information. However, this study only illustrates a potential use of the data mining method.

In the medical field accuracy in prediction of the diseases is the most important factor. In the analysis of data mining techniques, ANN classifier gives 92.83% of highest accuracy. Since the diabetes is a chronic disease, it has to be prevented before it affects people. In future, the proposed work can be further enhanced and expanded for the disease prediction. For instance, the feature used in this project can incorporate other medical attributes. It can also consider using other data mining techniques, like Time Series, Clustering and Association Rules. Moreover, continuous data can be used instead of discrete data as well.

CHAPTER- 10 BIBLOGRAPHY

1. Devi, M. Renuka, and J. Maria Shyla. "Analysis of Various Data Mining Techniques to Predict Diabetes Mellitus." International Journal of Applied Engineering Research 11.1 (2016): 727-730.
2. Berry, Michael 1., and Gordon Linoff. Data mining techniques: for marketing, sales, and customer support. John Wiley & Sons, Inc., 1997.
3. Witten, Ian H., et al. Data Mining: Practical machine learning tools and techniques. Morgan Kaufmann, 2016.
4. Emoto, Takuo, et al. "Characterization of gut microbiota profiles in coronary artery disease patients using data mining analysis of terminal restriction fragment length polymorphism: gut microbiota could be a diagnostic marker of coronary artery disease." Heart and vessels 32.1 (2017): 39-46.
5. Giri, Donna, et al. "Automated diagnosis of coronary artery disease affected patients using LDA, PCA, ICA and discrete wavelet transform." Knowledge-Based Systems 37 (2013): 274-282.
6. Fatima, Maheshwar, and Maruf Pasha. "Survey of Machine Learning Algorithms for Disease Diagnostic." Journal of Intelligent Learning Systems and Applications 9.01 (2017): 1.
7. Huang, Guang-Bin, Qin-Yu Zhu, and Chee-Kheong Siew. "Extreme learning machine: theory and applications." Neurocomputing 70.1 (2006): 489-501.
8. Huang, Guang-Bin, Qin-Yu Zhu, and Chee-Kheong Siew. "Extreme learning machine: theory and applications." Neurocomputing 70.1 (2006): 489-501.
9. Tiwari, Mukesh, Jan Adamowski, and Kazimierz Adamowski. "Water demand forecasting using extreme learning machines." Journal of Water and Land Development 28.1 (2016): 37-52.
10. Boyd, C. R.; Tolson, M. A.; Copes, W. S. (1987). "Evaluating trauma care: The TRISS method. Trauma Score and the Injury Severity Score". The Journal of trauma. 27 (4): 370 - 378. doi: I 0.1097/00005373-198704000-00005. PMID 3106646.